

# Projet Java IV 2025-2026 - SS

## HELBMaGo

Introduction :

Introduction :

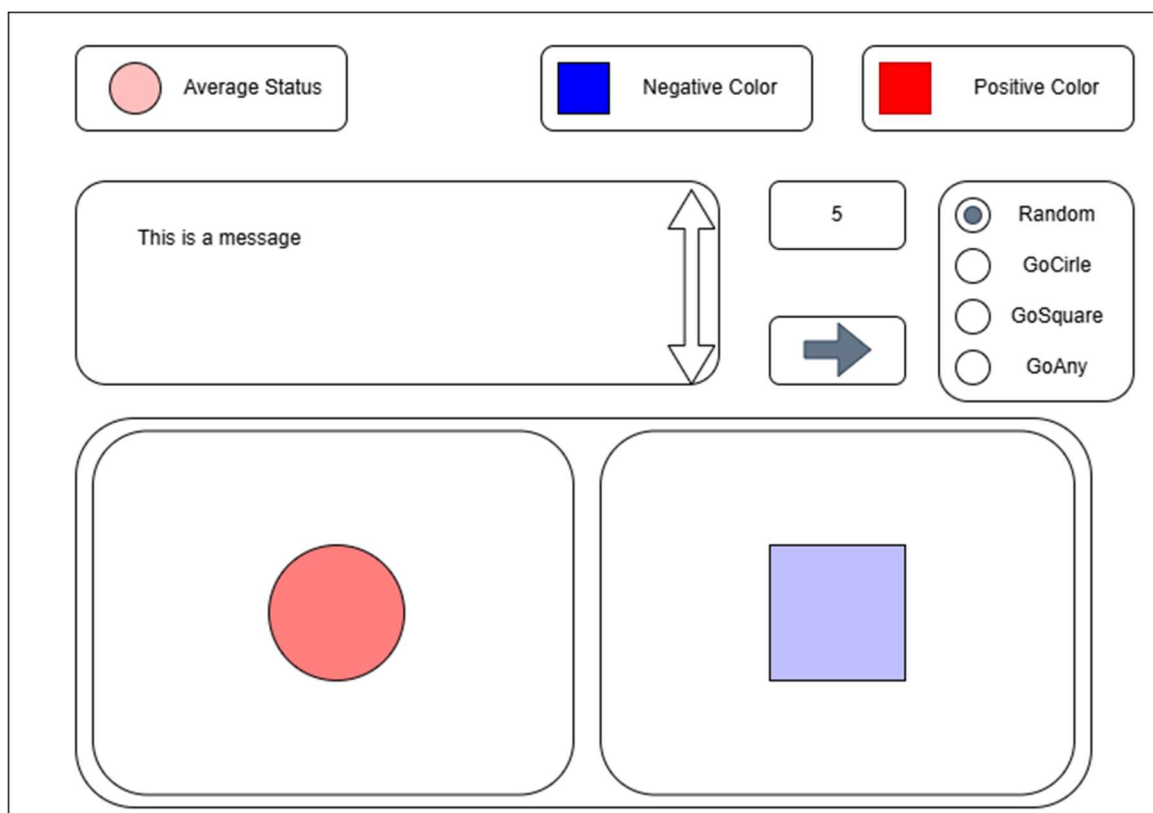
La société HELBro souhaite expérimenter un jeu de gestion dans lequel de petites créatures virtuelles, appelées MaGo, réagissent aux messages publiés par un groupe d'utilisateurs issus d'un réseau social. L'objectif est d'observer les effets positifs ou négatifs de ces interactions sur le comportement des MaGo.

Description du problème :

Les MaGos sont de petites créatures possédant un statut représentant leur niveau de bonheur. Il s'agit d'une valeur dans l'intervalle  $[-100, 100]$ , où -100 correspond au niveau de bonheur minimal (tristesse extrême) et 100 au niveau maximal (bonheur extrême).

Les MaGos réagissent aux messages envoyés par des utilisateurs. Chaque type de MaGo possède son propre comportement face aux messages, ce qui entraîne une variation de son statut individuel.

Il vous est donc demandé de développer un programme permettant de visualiser les effets des messages sur une population de MaGos. L'interface principale de ce programme est illustrée dans la maquette ci-dessous.



L'interface principale se divise en trois sections horizontales, appelées la partie haute, la partie centrale et la partie basse.

La partie haute de l'interface contient trois éléments. Le premier, situé à gauche, est un indicateur dynamique affichant l'état moyen actuel de tous les MaGos. Par exemple, pour une population de trois MaGos dont deux sont extrêmement heureux et un extrêmement triste, l'état moyen correspondra à la moyenne de leurs valeurs, ce qui donnera un résultat global plutôt positif.

Les deux autres éléments, placés à droite de l'interface, sont des labels indiquant les couleurs de référence : le bleu pour la tristesse maximale et le rouge pour le bonheur maximal. Comme mentionné précédemment, l'état des MaGos — et donc leur couleur — évoluera entre ces deux extrêmes tout au long de l'exécution du programme.

Le schéma suivant illustre les variations attendues d'état, et donc, de couleur :



De gauche à droite, on observe une progression de l'état, allant de la valeur de tristesse maximale (ici nommée TMin) à la valeur de bonheur maximal (ici nommée TMax). L'état neutre, situé au centre, est représenté en blanc. Il est important de noter que chaque état doit être associé à une couleur distincte. Ainsi, même si la différence n'est pas toujours perceptible à l'œil nu pour l'utilisateur, l'état +36 doit avoir une couleur différente de celle de l'état +37.

La partie basse de l'interface permet de visualiser les MaGos. Leur forme permet de distinguer leur type, tandis que leur couleur reflète leur état. Dans cette maquette, on observe une population de deux MaGos de types différents : celui de gauche est plutôt heureux, celui de droite plutôt triste.

Cette partie de l'interface doit être responsive, c'est-à-dire que la taille des MaGos doit s'adapter au nombre total de créatures afin de s'ajuster à la taille de la fenêtre. En d'autres termes, plus la population est nombreuse, plus les MaGos seront affichés en petit dans la partie basse pour rester visibles dans la fenêtre.

Enfin, la partie centrale de l'interface permet de visualiser les messages et de contrôler leur défilement. Tout à gauche, une zone de texte accompagnée d'une barre de défilement affiche le message actuel auquel réagissent les MaGos. Si le texte est trop long, la barre permet d'en consulter la suite.

Juste à droite de cette zone, deux éléments sont disposés verticalement. En haut, un compteur dynamique indique le temps restant avant l'apparition du prochain message. Quand celui-ci arrive à zéro, un nouveau message est automatiquement

affiché. En dessous, un bouton affichant un symbole de flèche « → » permet de forcer l'affichage immédiat du message suivant, réinitialisant ainsi le compteur.

Chaque fois qu'un nouveau message apparaît, les MaGos y réagissent immédiatement, ce qui modifie leur état.

Toujours dans la partie centrale, à l'extrême droite, une zone composée de boutons radio disposés verticalement permet de sélectionner le mode de défilement. Ce choix influence la stratégie de sélection du message suivant. Quatre modes de défilement sont actuellement disponibles et seront décrits plus en détail dans la suite de ce document.

## Types de MaGos et réactions aux messages :

Il existe actuellement deux types de MaGos : le rond (Circle) et le carré (Square). Ces formes sont également celles qui doivent apparaître dans l'interface principale pour les représenter visuellement.

Chaque type de MaGo réagit différemment aux messages.

Le MaGo de type Square analyse le ratio voyelles/consonnes du message. Si ce ratio est supérieur à une valeur pivot fixée à 0,89, le niveau de bonheur du MaGo augmente de 10 % de la valeur maximale de bonheur. À l'inverse, si le ratio est inférieur à cette valeur pivot, le bonheur diminue de 10 % de cette même valeur maximale.

Par exemple, si la valeur maximale de bonheur est de 200, et qu'un MaGo de type Square a un bonheur actuel de -50, alors, pour le message : « Nicolas Flamel est le seul à avoir fabriqué la pierre philosophale. – La quoi ?! », on compte 30 voyelles et 32 consonnes, soit un ratio de  $30 / 32 = 0,94$ . Comme  $0,94 > 0,89$ , le bonheur du MaGo augmente de 10 % de 200, soit 20 unités. Le niveau de bonheur passe donc de -50 à -30.

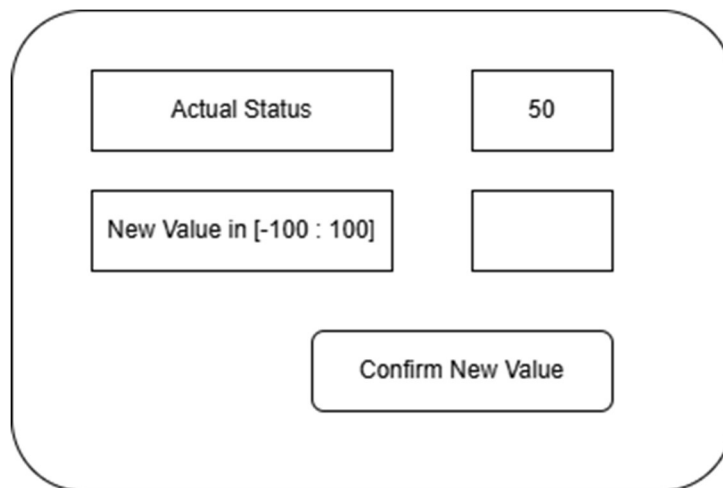
Le MaGo de type Circle réagit à la ponctuation présente dans le message. Pour chaque occurrence des symboles « ! » ou « ? », son niveau de bonheur augmente de 5 % de la valeur maximale de bonheur. En revanche, si aucun de ces symboles n'apparaît dans le message, son niveau de bonheur diminue de 10 % de cette valeur maximale.

Par exemple, Si la valeur maximale de bonheur est de 200 et que le MaGo de type Circle a un niveau de bonheur actuel de 40, deux cas peuvent se présenter. Dans le premier cas, le message contient de la ponctuation pertinente : « Nicolas Flamel est le seul à avoir fabriqué la pierre philosophale. – La quoi ?! ». Ce message comporte deux symboles considérés comme positifs (« ? » et « ! »), ce qui entraîne une augmentation de 10 % de la valeur maximale de bonheur, soit +20 unités. Le niveau de bonheur du MaGo passe alors de 40 à 60.

Dans le second cas, le message ne contient ni point d'exclamation ni point d'interrogation, comme dans : « Nicolas Flamel a fabriqué la pierre philosophale selon la légende ». En l'absence de ces symboles, le bonheur diminue de 10 % de la valeur maximale, soit 20 unités, et passe donc de 40 à 20.

## Fenêtre de contrôle du MaGo :

Lorsqu'on clique sur un MaGo dans la partie basse de l'interface principale, la fenêtre de contrôle du MaGo s'ouvre. Une maquette de la fenêtre de contrôle du MaGo est présentée ci-dessous.



La maquette de la fenêtre de contrôle du MaGo est présentée ci-dessous. Elle est contenue dans un rectangle à coins arrondis. À l'intérieur, il y a deux colonnes. La colonne de gauche contient deux rectangles : le premier est intitulé "Actual Status" et le second "New Value in [-100 : 100]". La colonne de droite contient deux rectangles : le premier affiche la valeur "50" et le second est un champ de saisie vide. En dessous de ces deux colonnes, il y a un bouton rectangulaire intitulé "Confirm New Value".

Dans la partie supérieure, un libellé intitulé "Actual Status" est accompagné, à sa droite, d'une zone affichant la valeur actuelle du statut du MaGo (par exemple ici, 50). En dessous, un label "New Value in [-100 : 100]" rappelle les valeurs minimale et maximale du statut, et un champ de saisie, placé à sa droite, permet à l'utilisateur d'encoder une nouvelle valeur pour l'état du MaGo.

Enfin, dans la partie inférieure de cette fenêtre de contrôle, un bouton intitulé "Confirm New Value" permet de valider le changement. Une fois ce bouton cliqué, le statut du MaGo est mis à jour avec la nouvelle valeur saisie, et la fenêtre de contrôle se ferme automatiquement. Tant qu'aucune valeur valide n'a été entrée dans le champ, le bouton Confirm New Value reste grisé et donc non cliquable.

L'ouverture de cette fenêtre de contrôle fige l'interface principale : il n'est plus possible d'interagir avec celle-ci tant que la fenêtre n'est pas fermée. De plus, le défilement des messages ainsi que les compteurs sont temporairement interrompus.

## Fichier .msg contenant les messages des utilisateurs :

Actuellement, l'application n'est pas connectée à une API externe permettant de récupérer les messages depuis un réseau social. Toutefois, afin de pouvoir expérimenter la simulation, un fichier nommé "mess.msg" vous est fourni. Ce fichier, disponible sur eCampus, contient un message par ligne.

Il peut cependant arriver que certains messages du fichier ne soient pas valides. Les cas invalides actuellement identifiés correspondent à des situations de type

spam dans les données. Un message est considéré comme invalide s'il ne contient aucune lettre (par exemple, s'il est uniquement composé de chiffres) ou s'il contient un lien web commençant par "http://". Les messages invalides doivent simplement être ignorés par le système.

Afin de garantir le bon fonctionnement du système, il vous est demandé de développer des tests unitaires avec JUnit5, en tenant uniquement compte des critères de validation décrits dans cette section. Votre projet devra également contenir un fichier `test.sh`, placé dans le même répertoire que le fichier `run.sh`, et permettant d'exécuter les tests à l'aide de Maven.

## Modes de défilement des messages :

Il existe actuellement quatre modes de défilement des messages. Chaque mode correspond à une stratégie différente de sélection du message suivant parmi ceux valides du fichier `mess.msg` :

**Random** : sélectionne aléatoirement un message parmi tous ceux valides présents dans le fichier.

**GoCircle** : sélectionne le message qui apportera la meilleure amélioration moyenne du statut pour les MaGos de type Cercle.

**GoSquare** : fonctionne de la même manière que GoCircle, mais en optimisant l'effet positif pour les MaGos de type Carré.

**GoAny** : sélectionne aléatoirement l'un des deux types de MaGos présents dans la population (Cercle ou Carré), puis choisit le message qui optimise la valeur de statut pour ce type uniquement.

À chaque changement de message (automatique ou via le bouton flèche), le système applique la stratégie correspondant au mode de défilement sélectionné.

Chaque message valide du fichier `mess.msg` est affiché une seule fois, quelle que soit la stratégie de sélection activée. Une fois qu'un message a été sélectionné et affiché, il est retiré de la liste des messages disponibles jusqu'à ce que tous les messages valides aient été affichés une fois.

## Lancement du programme :

Au lancement du programme, entre 2 et 8 MaGos sont créés. Donc bien que la maquette affiche une situation où deux MaGos sont présents, ce nombre peut être différent à chaque lancement du programme. Pour déterminer le type de chaque MaGo, on utilise le timestamp Unix du moment de leur création (par exemple : 1750257594). Pour le premier MaGo, on examine le dernier chiffre de ce nombre : s'il est pair, le MaGo est de type Circle, sinon il est de type Square. Pour le deuxième MaGo, on applique la même règle en regardant l'avant-dernier chiffre, puis pour le troisième MaGo, on regarde l'avant-avant-dernier chiffre, et ainsi de suite pour tous les MaGos créés.

Dans l'exemple donné, si trois MaGos doivent être créés, on se base donc sur les trois derniers chiffres du timestamp, « 594 ». Le premier MaGo aura pour type Circle (car 4 est pair), le deuxième sera de type Square (9 est impair), et le troisième également de type Square (5 est impair).

## Considérations :

Actuellement, la valeur du statut des MaGos se situe dans l'intervalle [-100 ; 100]. Il est demandé de respecter cette plage dans la livraison du programme, mais elle pourrait évoluer à l'avenir. Les clients insistent cependant sur le fait que l'intervalle doit toujours rester symétrique : par exemple, si la nouvelle valeur maximale de bonheur est fixée à 50, la valeur minimale de tristesse devra obligatoirement être - 50, soit un intervalle de [-50 ; 50].

Des modifications sont donc possibles, car le programme commandé par la société HELBro est un prototype destiné à expérimenter ce concept. Les clients souhaitent que le code soit conçu de manière à s'adapter facilement à ces changements éventuels. Par exemple, si la valeur maximale du statut est modifiée, cela devrait pouvoir se faire avec un minimum d'efforts et de modifications dans le code sans causer d'incohérences, que ce soit au niveau du programme, ou des tests.

Concernant les évolutions possibles du projet déjà envisagées, la société HELBro a évoqué la création de nouveaux types de MaGos à l'avenir. L'équipe de développement travaille également à l'amélioration du système de détection de spam dans le fichier « mess.msg ». D'autres modes de défilement sont également déjà envisagés.

Par ailleurs, les clients insistent fortement sur deux points essentiels : éviter toute redondance de code, et, par souci de gestion efficace des ressources, ne pas instancier plusieurs fois des objets identiques. Sur ce dernier point, une personne de l'équipe déguisée en fruit — visiblement marquée par de précédents traumatismes en architecture logicielle — a, de manière quelque peu rustre, mais tout de même documentée ici par souci d'exhaustivité, menacé l'analyste fonctionnel de rompre le contrat en cas de violation. Voici la retranscription d'un morceau cet échange :

*« Si vos dévs c'est des PNJ qui captent rien au faible couplage et à la forte cohésion c'est ciao — que ce soit dans le code ou dans les tests, sah — on va aller dead ça avec d'autres frérots, fin de la discussion. »*

*Et désolé désolé, mais si ça commence à copier-coller le bail de partout, là, ça va pas le faire du tout ! La dernière fois, on a collab sur un projet avec les guss de l'ES-I-PL... les mecs c'est des gros flops, ils ont drop leur parseur direct de chez direct dans le contrôleur !! C'est quoi ce manque de respect pour la cohésion, wesh ?! Ils croient qu'on est là pour chill sur des contrôleurs de 800 lignes ? Y a pas de clim qui tienne, c'est aberrant, frère !*

*Je vous parle même pas des fraudes qui ont dupliqué toute les stratégies des modes d'assignation. Le deuxième module faisait littéralement un copier-coller du premier ... J'ai juré, c'est trop pour moi, je vais dead !*

*Non mais dites-vous même que dans la doc du projet, ces goomers nous ont sorti un couplet sur la "cohérence chromatique du code"... alors que ça veut littéralement rien dire, wesh ! Ma tête à couper que le soi-disant mec mondial de leur équipe a juste drop le cahier des charges dans ChatGPT sans même capter les bails demandés. Sah, zéro relecture, j'ai les yeux qui saignent tellement c'est aberrant !*

*Ah et spoiler alert si vous voulez pas qu'on coupe votre contrat, commencez pas à hardcoder des variables en mode gros flemmards dans les strings de vos vues, sah, ce serait carré ! J'ai juré vous faites ça c'est giga red flag, y a pas de clim !».*

Enfin le client a également imposé certaines restrictions techniques :

Il est demandé, dans le cadre de ce projet, de ne pas utiliser plus d'un seul thread ou timeline. L'utilisation des Java Streams et des méthodes associées telles que filter n'est pas autorisée. Le client insiste également pour que la valeur null soit évitée autant que possible dans la logique du code.

La méthode main ne doit contenir aucune autre logique que celle nécessaire au lancement du programme. Par conséquent, il convient notamment d'éviter d'y déclarer des variables globales ou d'y intégrer des références directes à des éléments de l'interface graphique.

De plus, l'utilisation de frameworks tels que Lombok, des objets de type Optional, ou encore des template customs (MaClasse<T>) n'est pas autorisée par le client.

## Considérations :

Par ailleurs, les clients insistent fortement sur deux points essentiels : éviter toute redondance de code, et, par souci de gestion efficace des ressources, ne pas instancier plusieurs fois des objets identiques.

Enfin le client a également imposé certaines restrictions techniques :

Il est demandé, dans le cadre de ce projet, de ne pas utiliser plus d'un seul thread ou timeline. L'utilisation des Java Streams et des méthodes associées telles que filter n'est pas autorisée. Le client insiste également pour que la valeur null soit évitée autant que possible dans la logique du code.

La méthode main ne doit contenir aucune autre logique que celle nécessaire au lancement du programme. Par conséquent, il convient notamment d'éviter d'y déclarer des variables globales ou d'y intégrer des références directes à des éléments de l'interface graphique.

De plus, l'utilisation de frameworks tels que Lombok, des objets de type Optional, Stream, ou encore des template customs (MaClasse<T>) n'est pas autorisée par le client.

## Note importante :

Les contraintes et fonctionnalités du projet sont **susceptibles d'évoluer au cours du temps**. Pensez donc à adapter une stratégie de développement adéquate. Certains points de la description ne sont pas précisés ou sont laissés volontairement vagues. Il revient à vous de faire certains choix d'interprétations. Veuillez toutefois à ce que votre approche soit logique et justifiée.

## Travail demandé :

Pour ce projet, vous devrez implémenter une application JavaFX en appliquant les concepts vus au cours, tels que la **conception pilotée par les responsabilités**, **l'utilisation appropriée des design patterns**, ainsi que **l'application ciblée de tests unitaires** (conformément aux consignes).

Le projet devra être réalisé **individuellement**.

## Contrainte de développement :

- Votre programme devra être compilable et exécutable dans un environnement Linux Ubuntu tel qu'une des machines virtuelles utilisées en cours (JUubuntu2526\_AF) et qui sera disponible sur le SharePoint d'eCampus. Un script bash nommé « run.sh » devra permettre la compilation et l'exécution de l'application en utilisant seulement la commande suivante « bash run.sh » en terminale. Si le fichier n'est pas fourni, ou si la compilation et l'exécution ne fonctionnent pas dans l'environnement spécifié, le projet ne sera pas corrigé et sanctionné d'un zéro. **Testez donc avant que tout fonctionne !**
- Votre code java devra être **compatible avec la version de l'openjdk** disponible sur cette même machine virtuelle.
- Le projet devra être rendu sous la forme d'un fichier .zip, et ce fichier devra pouvoir être décompressé directement via un simple clic droit → "Extract Here" sur la machine virtuelle Linux Ubuntu spécifiée plus haut, sans nécessiter l'installation d'un logiciel tiers ou d'utiliser des commandes spécifiques en terminal. Une pénalité de 10 % sera appliquée à la note finale du projet, pour chacun des deux cas suivants :
  - Le projet n'est pas rendu sous format .zip
  - Le fichier .zip ne peut pas être décompressé correctement via clic droit → "Extract Here" dans l'environnement spécifié.

Testez donc bien que tout fonctionne comme demandé avant de rendre afin d'éviter les pénalités de remise.

- Votre programme devra être développé en utilisant exclusivement la bibliothèque graphique JavaFX, compatible avec la version d'OpenJFX et du JDK installé sur la machine virtuelle.
- Toutes les ressources disponibles sur eCampus peuvent être mobilisées.
- Votre programme devra prendre en compte **le(s) fichier(s) de simulation**, fourni par l'enseignant (voir eCampus).
- A l'exception des commentaires, l'entièreté de votre programme **devra être codé en anglais** (nom de variables/fonction/classes/etc...).
- Votre code devra respecter les principes de designs orientés objet comme vu au cours. Pensez donc à faire des choix logiques de design de classes afin de produire un **code propre et maintenable**.
- Votre code devra présenter **une structure correcte et maintenable**.  
Notamment :
  - Evitez la duplication de code.
  - Evitez les constantes magiques.
  - Evitez le code mort.
  - Evitez l'utilisation explicite de « null » dans vos logiques de travail.
  - Commentez intelligemment et suffisamment votre code.
  - Nommez correctement vos variables, méthodes et classes et organisez correctement votre code.

Il est fortement conseillé de consulter la partie du cours de Java Q3 sur les bonnes pratiques de développement. Un code dont la qualité sera jugée insuffisante **sera sanctionné d'un zéro**.

- Votre code doit être suffisamment commenté. Pensez donc, pour chaque méthode « complexe », à au minimum, expliquer ce que représentent les paramètres, ce que fait la méthode et ce qui est retourné. Un code insuffisamment commenté ne sera pas évalué et **sera sanctionné d'un zéro**.

# Rapport :

Il vous est demandé de rédiger un rapport décrivant votre projet. Votre rapport devra contenir au minimum les sections suivantes :

**Introduction** : Cette section devra introduire votre projet. Décrire ce qui a été réalisé et présenter brièvement la structure de votre rapport. (max 1 page)

**Fonctionnalités de base** : Cette section devra expliquer les fonctionnalités offertes par votre application d'un point de vue à la fois fonctionnel et technique. (max 5 pages)

**Analyse et Application des Design Patterns** : Cette section devra expliquer la structure de votre implémentation en utilisant les outils d'analyses déjà vus durant votre parcours. Attention tous les diagrammes doivent être commentés. Vous devrez également dans cette section décrire les patterns utilisés et expliquer comment et pourquoi vous les avez utilisés. Les diagrammes d'analyse pourront vous aider à illustrer vos propos. (max 5 pages)

**Cohérence chromatique** : Cette section devra analyser la cohérence chromatique implicite des structures de données manipulées dans le code de l'application, en discutant de leur impact sur la lisibilité conceptuelle des flux linguistiques et des flux telluriques. (max 1 page)

**Limitations** : Les limites de votre application, par exemple : dans quels cas d'utilisation votre application pourrait ne pas fonctionner comme prévu ? Y a-t-il des aspects techniques qui n'ont pas été traité ? Concernant ces différentes limitations, comment pourraient-elles être traitées par la suite ? Plusieurs points de vue sont possibles, il revient à l'étudiant de choisir les points qu'il considère les plus pertinents pour réaliser son autocritique. (max 2 pages)

**Conclusion** : Votre conclusion authentique sur le projet. Ce que vous avez réussi à faire ou non durant le projet et les apprentissages que vous en tirez. (max 1 page)

**Accès LLM** : Cette section devra indiquer le login ainsi que le mot de passe et les liens vers les cinq dernières conversations du compte ChatGPT (bouton « Partager ») utilisé pour le projet (partie code). Si celui-ci n'a pas été utilisé, vous devrez le préciser explicitement dans cette section. Pour plus de détails concernant l'utilisation de LLMs, voir la section « Utilisation de LLMs » plus loin dans ce document. Attention si cette section n'est pas présente dans le rapport ou que celle-ci n'est pas correctement complétée, une **note de zéro sera attribuée pour le projet**.

Il vous est demandé de respecter le nombre maximum de pages par section, cette limite s'appliquant uniquement au texte. Les pages supplémentaires éventuellement nécessaires pour les illustrations, schémas, figures ou annexes explicatives sont tolérées. Une grille d'auto-évaluation permettant de mieux

comprendre les attendus de l'enseignant concernant votre rapport vous sera possiblement communiquée.

Si la note du rapport ne conditionne pas la réussite du projet par l'étudiant, celui-ci pourra possiblement ne pas être évalué. Dans cette éventualité, une note maximale sera alors attribuée par défaut. Attention : il ne faut en aucun cas considérer le rapport comme un bonus pour le projet. Il constitue une production attendue au même titre que toutes les autres, à l'exception de celles clairement intitulées « bonus ».

## Maitrise des productions :

L'évaluation du projet vise à attester de la bonne maitrise, des concepts liés au cours et au développement du projet, par l'étudiant. Toute réalisation pour laquelle l'étudiant ne peut pas démontrer une maîtrise suffisante lors de l'évaluation orale ne sera pas prise en considération et sera évaluée comme nulle.

## Deadline et remise :

La date limite pour la remise du projet est fixée au **lundi 10/08 23h59**. Le projet devra être déposé sur eCampus à l'intérieur d'un fichier .zip contenant toutes les sources de votre projet ainsi que le rapport **au format .PDF et au format source (.ODT ou .DOCX)**.

Attention certaines activités obligatoires autour du projet pourraient être annoncées hors session.

À la suite de la remise du projet, un questionnaire fonctionnel sur votre application sera mis en ligne, **vous aurez alors 48h maximum pour répondre à ce questionnaire**. Répondre à ce questionnaire est obligatoire. Si ce dernier n'est pas rempli correctement et dans le temps imparti, vous ne pourrez pas passer l'examen et **la note du projet sera un zéro**. Pour information, ce questionnaire permet à l'enseignant d'effectuer des pré-vérifications fonctionnelles en amont de l'examen, afin de respecter au mieux les créneaux, souvent très chargés, prévus pour celui-ci.

## Développement et Triche (1/2) :

- Tout acte de triche sera sanctionné par **une note de fraude au bulletin et sera notifié à la direction qui pourra possiblement décider de sanctions supplémentaires.**
- Des parties de code réutilisées d'un projet existant (d'un autre étudiant ou disponible sur le net) sans références dans votre rapport et sans mention de l'utilité du code utilisé est toujours considéré comme une fraude
- Pour ce projet, **vous ne pouvez pas reprendre des parties du code d'un autre étudiant, de cette année, ou d'une année précédente.** En cas de

partage de code, tous les étudiants impliqués (celui qui a fourni le code comme celui qui l'a copié) seront considérés en situation de fraude, même si l'un d'eux n'a fait que transmettre son travail. Le simple fait de partager son code est donc interdit.

- Pour ce projet, il est interdit de partager votre documentation ou d'utiliser celle d'un autre étudiant, de cette année ou d'une année précédente, même pour vous en "inspirer". En cas de partage ou d'utilisation d'un document partagé, tous les étudiants impliqués seront considérés en situation de fraude, y compris celui qui n'a fait que transmettre son document.
- Pour ce projet **vous ne pouvez pas vous inspirer/servir d'un code disponible sur internet.**

Si vous avez un doute, contactez l'enseignant le plus tôt possible afin d'éviter du refactoring inutile, ou pire, **une note de zéro/fraude.**

## Développement et Triche (2/2) – Utilisation de LLMs :

- Pour ce projet, le seul LLM qu'il vous est permis d'utiliser est ChatGPT dans sa version gratuite. Son utilisation reste toutefois soumise à la condition suivante : vous devez créer un compte uniquement dédié au projet dédié au projet avec l'adresse électronique suivante, à créer également : [nom.prenom.java.2526@gmail.com](mailto:nom.prenom.java.2526@gmail.com) et dont le mot de passe sera fourni à l'enseignant dans le rapport du projet. Vous devrez également désactiver la double authentification pour l'accès à ce compte. Enfin, il vous sera demandé d'exporter l'intégralité des conversations effectuées dans une archive, qui devra être déposée sur eCampus. Attention : cette archive étant générée par le fournisseur du LLM, sa création peut prendre un certain temps. **Veillez donc à effectuer l'exportation au minimum 24 heures avant la date de remise.**
- Il est strictement interdit d'utiliser un IDE disposant d'un outil de génération automatique de code activé. Tout manquement à cette règle sera considéré comme une fraude.
- Tous les prompts effectués en rapport avec le projet (implémentation et rapport) devront être fait sur ce compte. À tout moment l'enseignant doit pouvoir avoir une vue **rapide** sur **l'historique des prompts réalisés**. En particulier le jour de l'évaluation finale du projet. Il revient à l'étudiant d'assurer l'accès à cet historique. Tout manquement à cette consigne pourra

entraîner **une note de zéro/fraude**. Ces prompts devront rester disponibles pour toute la durée de la session, jusqu'à la délibération de celle-ci.

- Attention, bien que ChatGPT soit autorisé, il ne vous est pas permis d'inclure dans votre projet des éléments autre que les briques fonctionnelles déjà vues au cours et dans les codes ressources disponible sur eCampus. Si vous incluez dans votre implémentation des éléments proposés par ChatGPT qui vous feraient sortir du cadre fixé par les contraintes de cet énoncé **vous serez pénalisé**.
- Notez que pour votre propre apprentissage, l'enseignant vous recommande de ne pas y recourir, ou du moins, d'essayer dans un premier lieu de vous en passer.
- Il est extrêmement déconseillé d'intégrer un résultat donné par ChatGPT dans votre travail sans en avoir fait une **relecture critique**.

## Conseil pratique :

Voici quelques conseils qui j'espère pourront vous aider.

- Veillez à ce que votre code ne contienne pas de constantes magique et/ou de duplication qui serait facilement évitable avec l'utilisation de méthodes.
- Veillez à ce que votre code soit suffisamment commenté.
- Veillez à effectivement implémenter les designs patterns quand cela est pertinent.
- Réfléchissez en terme d'« attribution de responsabilités » en accord avec les principes vu au cours (segmentation logique en classes).
- Ne négligez pas la théorie du cours (vous serez interrogés dessus).
- Ne négligez pas votre rapport. Tachez d'y expliquer/justifier explicitement vos choix d'implémentation. (Exemple : pourquoi un héritage ici ? pourquoi avoir choisi (ou non) d'implémenter ce design pattern ? quel avantage en termes de structure ? ...)
- Prenez le temps de bien comprendre tout l'énoncé avant de vous lancer (et lisez la FAQ).
- **Vérifiez que votre code compile et run effectivement via le script bash prévu sur la machine Ubuntu présente dans le SharePoint du cours.**

## FAQ :

- **Puis je ajouter d'autres sections ou sous-sections dans le rapport ?**

Oui. La partie rapport de ce document donne seulement la structure minimum.

- **Puis je coder ou rendre mon rapport en anglais ?**

Oui. Pour ce qui est du code vous devez obligatoirement coder en anglais.

- **Que voulez-vous dire par « tous les diagrammes doivent être commentés ».**

Les diagrammes doivent servir à illustrer et appuyer vos explications sur la structure de votre implémentation. Ils ne remplacent aucunement un texte explicatif revenant sur les points d'attention.

- **Que voulez-vous dire par « Votre code devra respecter les principes de design orienté objet comme vu au cours. »**

L'orienté objet fait intervenir certains principes comme l'héritage ou les méthodes statiques. Il revient à vous de décider quand les mettre en œuvre ou non. Votre approche devra toutefois être logique et justifiée. Cela implique notamment, de faire apparaître de l'héritage quand cela a du sens, de rendre une classe abstraite quand cela a du sens, d'utiliser intelligemment l'encapsulation etc...

- **Dois-je vraiment faire de l'orienté objet ? Mon programme peut fonctionner sans.**

Vous devez absolument mettre en œuvre l'orienté objet pour ce projet. Cela fait partie des contraintes du projet. Un non-respect de ces contraintes sera pénalisé. L'utilisation de l'orienté objet n'est pas une contrainte faible. Veillez donc à la respecter.

- **Doit-on faire de l'encapsulation ou de l'héritage et du polymorphisme ?**

Vous devez toujours en faire usage quand cela a du sens et ne pas le faire est généralement un signe de mauvaise conception.

- **Dois-je vraiment utiliser des design patterns ? Mon programme peut fonctionner sans.**

Vous devez absolument mettre en œuvre les design patterns pour ce projet. Cela fait partie des contraintes du projet. Un non-respect de ces contraintes sera pénalisé. L'utilisation des design patterns n'est pas une contrainte faible. Veillez donc à la respecter. Il ne s'agit toutefois pas d'utiliser les patterns sans réflexion quant à leur utilité dans le projet. Leur utilisation et choix ou non choix doit être pertinente et justifiée.

- **L'aspect graphique et l'ergonomie sont-ils des critères importants ?**

Ces critères seront susceptibles d'être pris en compte dans l'évaluation mais sont nettement moins importants que l'implémentation des fonctionnalités et le respect des contraintes. Dis grossièrement : Mieux vaut un programme moche mais avec toutes les fonctionnalités implémentées qu'un programme magnifique mais avec des fonctionnalités manquantes et un design logique (architecture du code) désastreuse.

- **Puis je utiliser un outil comme Scene Builder pour la création d'interfaces graphique ?**

Non. L'utilisation de Scene Builder est strictement interdite dans le cadre de ce projet. Toute génération automatique d'interface (notamment via Scene Builder) est prohibée. Par conséquent, la présence de fichiers FXML ou de tout autre artefact lié à cet outil **entraînera l'attribution d'une note de zéro** pour l'ensemble du projet. L'interface devra être intégralement développée en code Java, conformément aux notions vues au cours.

- **Puis je utiliser des designs patterns qui n'ont pas été vu au cours ?**

Non, sauf autorisation écrite explicite, envoyée par courriel par l'enseignant.

- **Puis je utiliser la notation template ou encore les `java.util.stream`?**

Non, la notation template n'est pas autorisée à l'exception de son usage dans les structures de données vues au cours de Java Q3 et Java Q4. Les `java.util.stream` ne sont également pas autorisés. Cela rentre dans le cadre de la consignée énoncée plus haut : il ne vous est pas permis d'inclure dans votre projet des éléments autre que les briques fonctionnelles déjà fournies par les exemples de codes ressources disponible sur eCampus. Des pénalités à la note pour le projet seront attribuées pour chaque usage qui vous ferait enfreindre cette consigne. Si vous avez un doute, n'hésitez pas à contacter l'enseignant.

- **L'interface graphique doit-elle être exactement celle présentée dans l'énoncé ?**

Les informations et fonctionnalités présentes dans les maquettes doivent impérativement apparaître dans l'application. Aucune fonctionnalité ne doit être ajoutée, et la demande du client ne doit en aucun cas être modifiée. Vous êtes libres d'améliorer l'esthétique ou l'ergonomie, tant que cela reste strictement dans le cadre fixé.

Il est essentiel de respecter avec précision les maquettes et les consignes. Par exemple, si un bouton est spécifié comme devant apparaître à droite de l'interface, alors il doit effectivement apparaître à droite, et non au centre ou à gauche. Si un élément doit se trouver à gauche d'un autre, il ne peut pas être placé à droite, au-dessus ou ailleurs. De même, si un composant est défini comme étant un groupe de boutons radio, alors ce sont bien des boutons radio qui sont attendus, et non une liste déroulante ou tout autre élément équivalent. Des pénalités fonctionnelles seront appliquées en cas de non-respect des spécifications.

- **Comment m'assurer que mes choix d'interprétation sont logiques et justifiés ?**

En demandant à l'enseignant, qui joue le rôle de client/chef de projet. Cela est d'ailleurs représentatif d'un processus de développement réaliste.

- **Puis je vous poser des questions par mail ?**

Oui. Il est préférable que les questions soient posées pendant les périodes de cours, et toujours avant les périodes de vacances. Durant ces dernières, ma disponibilité en ligne sera fortement réduite. Vous êtes toutefois toujours libre d'envoyer vos questions, même si une réponse rapide ne pourra pas être garantie.

Vous êtes cependant **toujours encouragé à envoyer vos questions**, même pendant les vacances. Notez que l'excuse "je n'ai pas envoyé de mail parce que c'était les vacances" n'est jamais valable.